

ДИАГРАММЫ СОСТОЯНИЙ (конечного автомата, машины состояний)

STATECHART DIAGRAM

Диаграмма состояний

Диаграмма состояний показывает конечный автомат, фокусируя внимание на потоке управления от состояния к состоянию.

Автомат описывает последовательности состояний, через которые проходит объект на протяжении своего жизненного цикла, реагируя на события, в том числе описание реакций на эти события.

Состояния

Состояние - это ситуация в жизни объекта, протяжении которой он удовлетворяет некоторому условию, осуществляет определенную деятельность или ожидает какого-то события.

Состояние может содержать список внутренних действий:

<метка> / <выражение действия>

Начальное (конечное) состояние не содержит внутренних действий. В этом состоянии находится объект в начальный (конечный) момент времени

Имя
состояния

NewState

entry/



Начальное
состояние



Конечное
состояние

Событие, переход

Событие - это спецификация существенного факта, который происходит во времени и пространстве. В контексте автоматов событие - это стимул, вызывающий срабатывание перехода.

Переход - это отношение между двумя состояниями показывающее, что объект, находящийся в первом состоянии, должен выполнять некоторые действия и перейти во второе состояние как только произойдет выделенное событие и будут выполнены заданные условия

Диаграмма конечного автомата

- является графом, который представляет некоторый конечный автомат

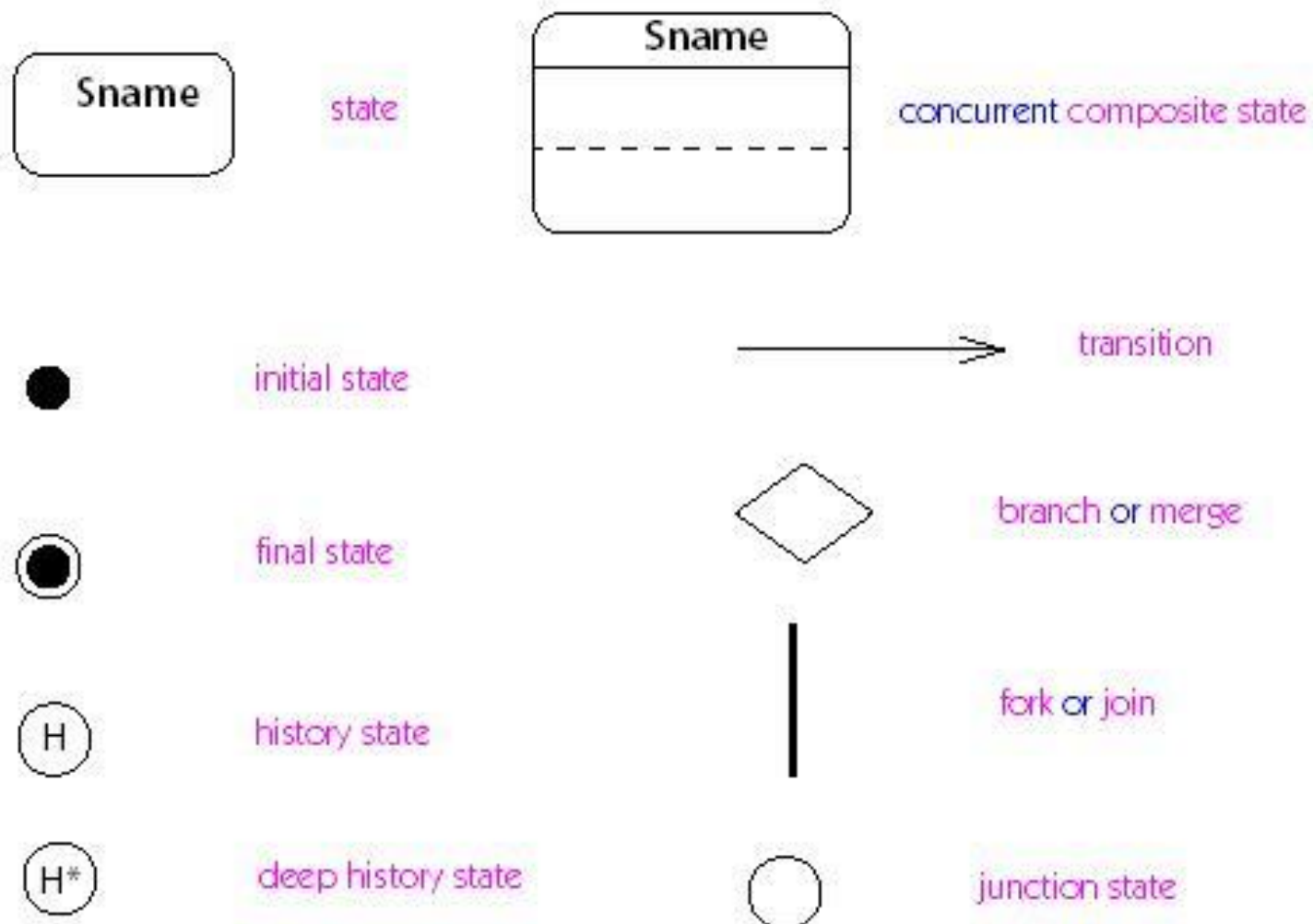
Конечный автомат (state machine) представляет собой некоторый формализм для моделирования поведения отдельных элементов модели или системы в целом.

Поведение (behavior) является спецификацией того, как экземпляр класса изменяет значения отдельных характеристик в течение своего времени жизни.

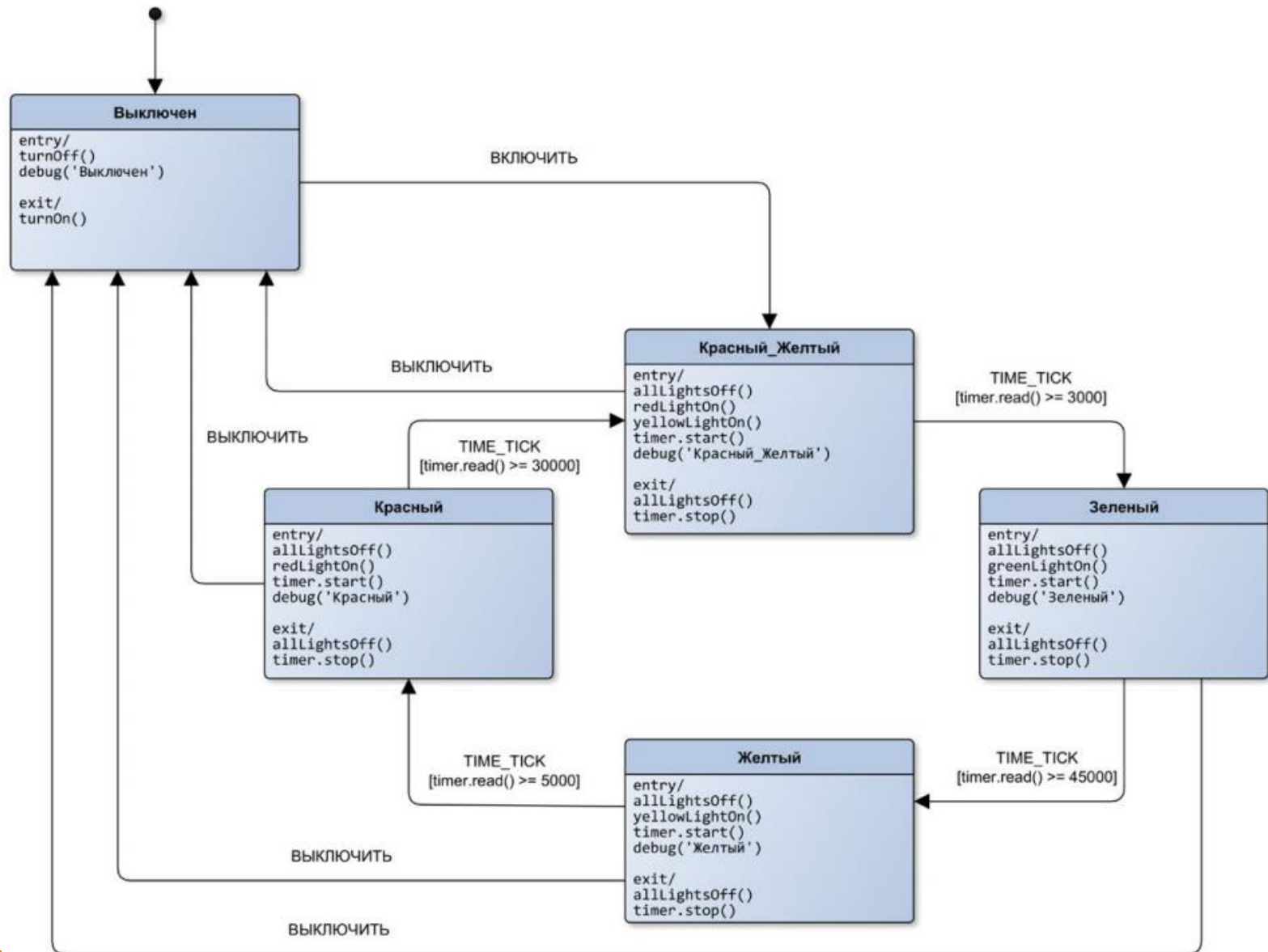
Состояние (state) – элемент модели поведения, предназначенный для представления ситуации, в ходе которой поддерживается некоторое условие инварианта.

Переход (transition) является направленным отношением между двумя состояниями, одно из которых является *источником (source state)*, а другое – *целевым (target state)*.

Основные обозначения



Пример диаграммы состояний



Спецификация перехода

Переход может быть помечен строкой текста, синтаксис которой определяется следующим выражением:

$\langle \text{переход} \rangle ::= \langle \text{триггер} \rangle [', ' \langle \text{триггер} \rangle]^* [[' \langle \text{сторожевое-ограничение} \rangle '] [' / ' \langle \text{выражение-деятельности} \rangle]]$

Здесь **<триггер>** представляет собой имя триггерного события. В качестве имени триггера могут использоваться имена операций с параметрами или без них.

Терм *<сторожевое-ограничение>* или *ограждающее условие* является логическим выражением, записанным в терминах параметров триггерного события, атрибутов и связей объекта контекста.

Терм *<выражение-деятельности>*, который иногда называют *выражением поведения*, выполняется, если и только если переход срабатывает

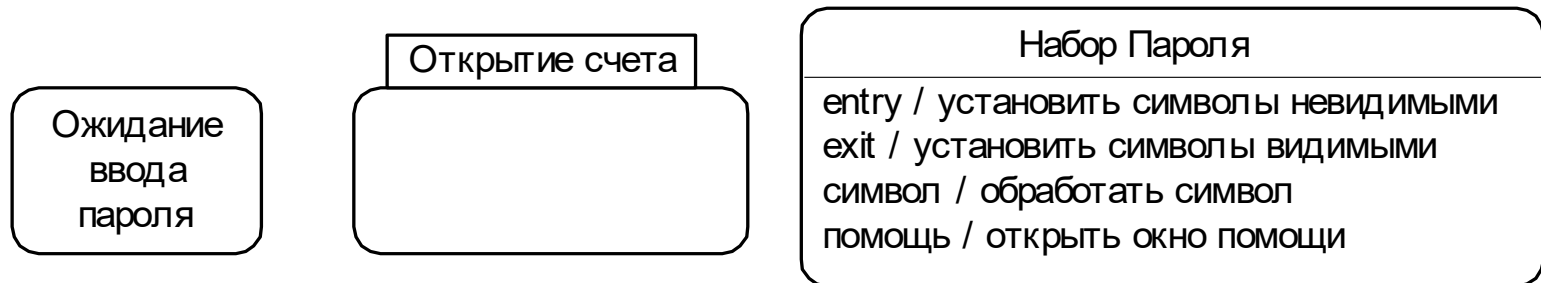
Простое состояние (simple state)

- называется состояние, которое не имеет внутренних регионов и подсостояний

На диаграмме конечного автомата допускается изображать состояния без имени, которые называются *анонимными* состояниями.

Все анонимные состояния считаются различными.

На одной диаграмме не следует показывать одно и то же именованное состояние дважды.



Простое состояние с внутренними действиями

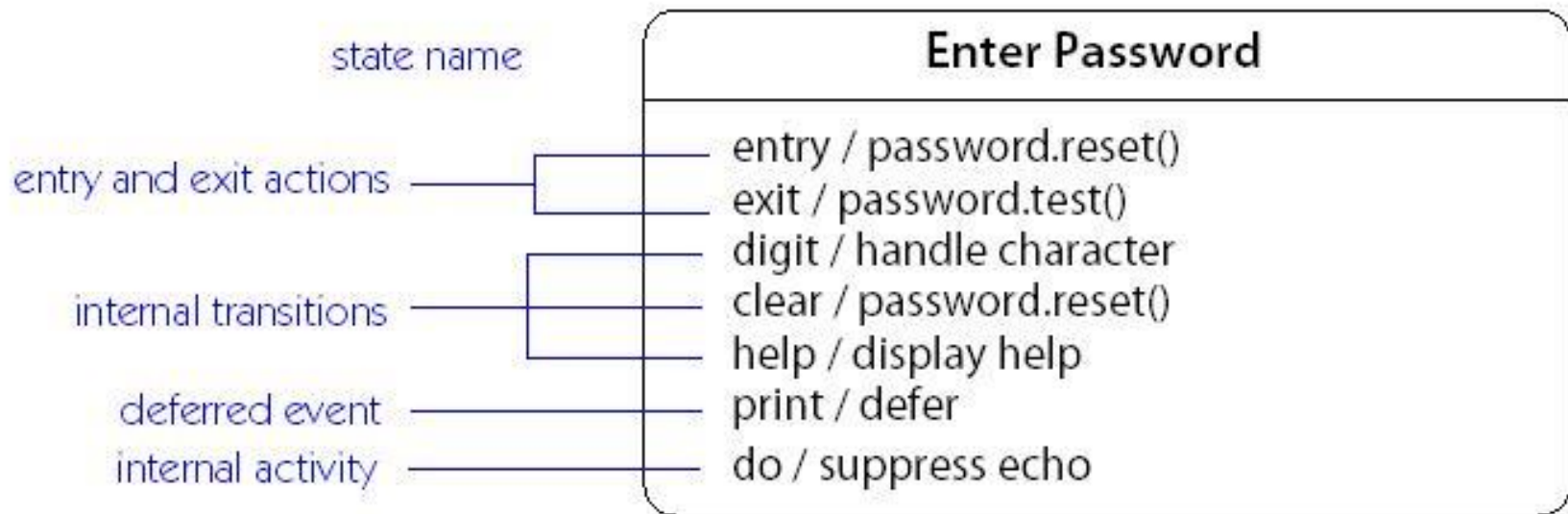
entry / action-expression

do / activity-expression

exit / action-expression

event-name / defer

include machine-name(argument_{list})_{opt}



Секция внутренней деятельности

entry - эта метка специфицирует входное поведение. Это поведение выполняется всякий раз, когда происходит вход в данное состояние независимо от перехода, позволившего достичь это состояние.

exit - эта метка специфицирует выходное поведение. Это поведение выполняется всякий раз, когда происходит выход из данного состояния независимо от перехода, который выводит из этого состояния.

do - эта метка специфицирует поведение, которое выполняется до тех пор, пока моделируемый элемент находится в данном состоянии, или до тех пор, пока не закончится выполнение деятельности, специфицированной соответствующим выражением.

Внутренние переходы и отложенные события

Внутренний (internal) переход выполняется без выхода из состояния и без повторного входа в состояние, в котором он определен.

Отложенные события сохраняются до тех пор, пока не будет достигнуто другое состояние, в котором эти события смогут инициировать некоторый переход.

Отложенное событие представляется отдельной строкой текста в соответствии с синтаксисом для триггера, после которой следует разделитель “/” (обратный слэш) и ключевое слово defer.

Спецификация триггерного перехода

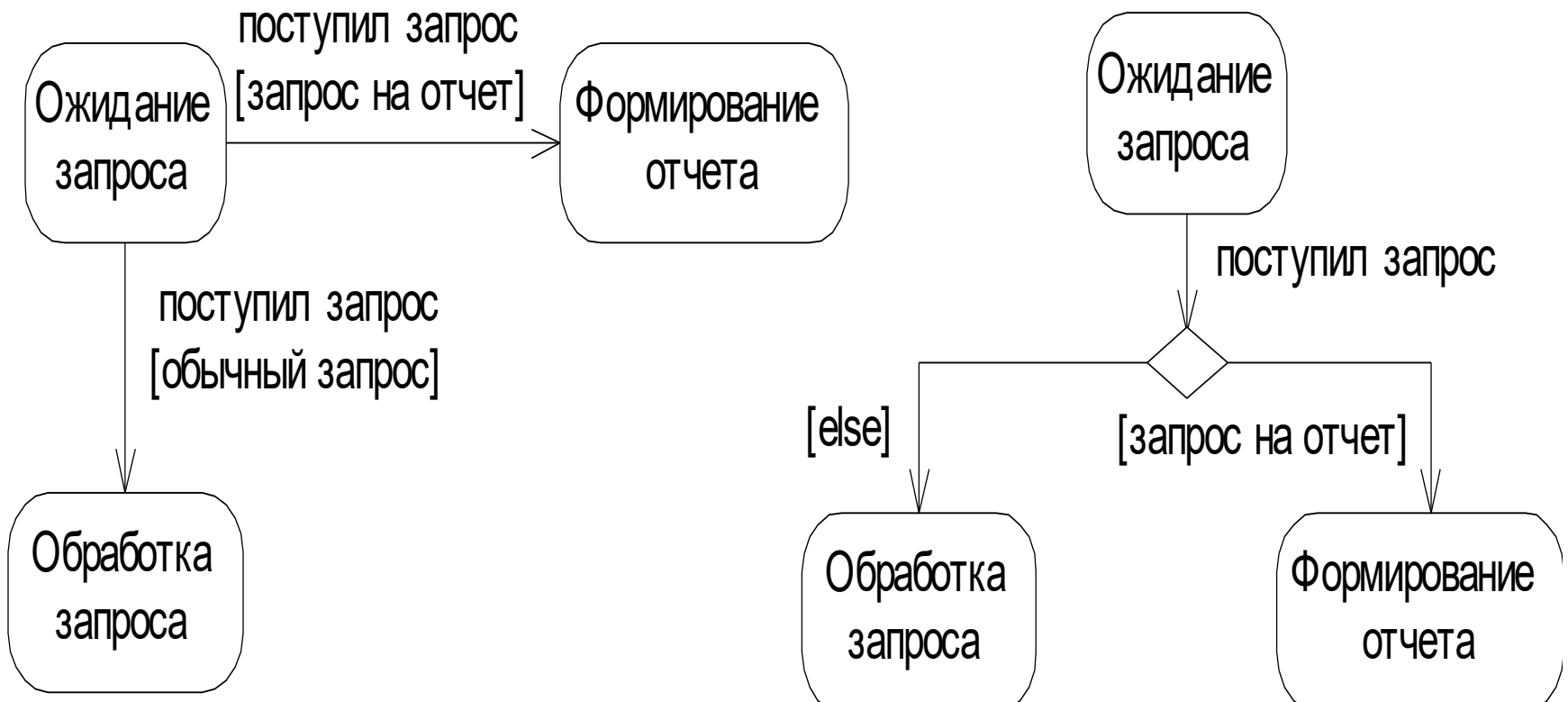
Событие (event) является спецификацией некоторых условий, которые оказывают влияние на поведение моделируемой сущности.

Триггер (trigger) устанавливает отношение события с поведением, которое может оказывать влияние на экземпляр классификатора.

name : opt event-name opt (parameter-list) opt [guard-condition] opt
/ action-list opt

Конфликтующие переходы

Два и более разрешенных перехода называются **конфликтующими** (*conflicting*), если все они выходят из одного и того же состояния



Псевдосостояния

– абстрактный элемент модели, который включает в себя различные типы вспомогательных вершин в графе конечного автомата

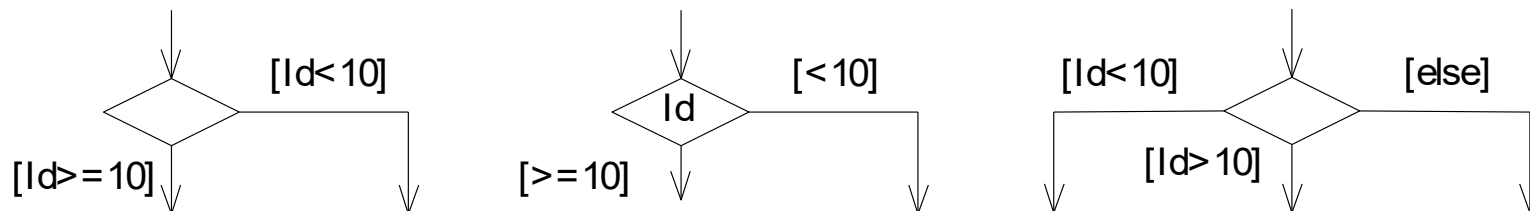
Начальное псевдосостояние (*initial pseudo state*) представляет вершину графа конечного автомата, которая по умолчанию является источником для начального перехода моделируемого поведения

Узел завершения (*terminate node*) является псевдосостоянием, вход в который означает завершение выполнения поведения конечного автомата

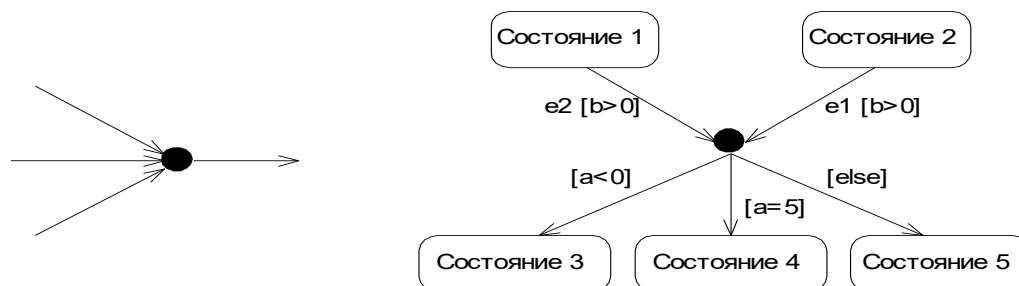
Финальное состояние (*final state*) – специальный вид состояния, предназначенное для моделирования завершения конечного автомата

Выбор и соединение

Псевдосостояние выбора (*choice pseudo state*) предназначено для моделирования нескольких альтернативных ветвей при реализации поведения конечного автомата.



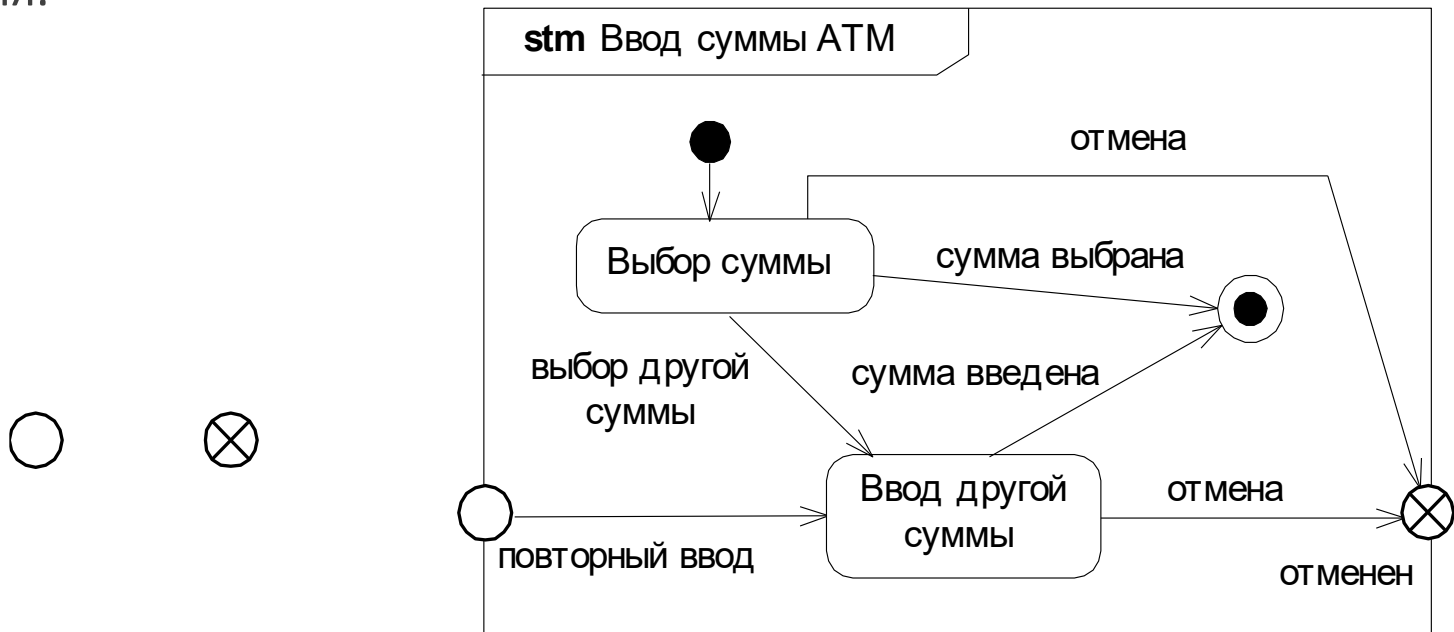
Псевдосостояние соединения (*junction pseudo state*) является вершиной со свободной семантикой, которая используется для соединения вместе нескольких переходов.



Точки входа и выхода

Точка входа (*entry point*) – псевдосостояние, предназначенное для моделирования входа в некоторый конечный автомат или композитное состояние.

Точка выхода (*exit point*) – псевдосостояние, предназначенное для моделирования выхода из некоторого конечного автомата или композитного состояния.



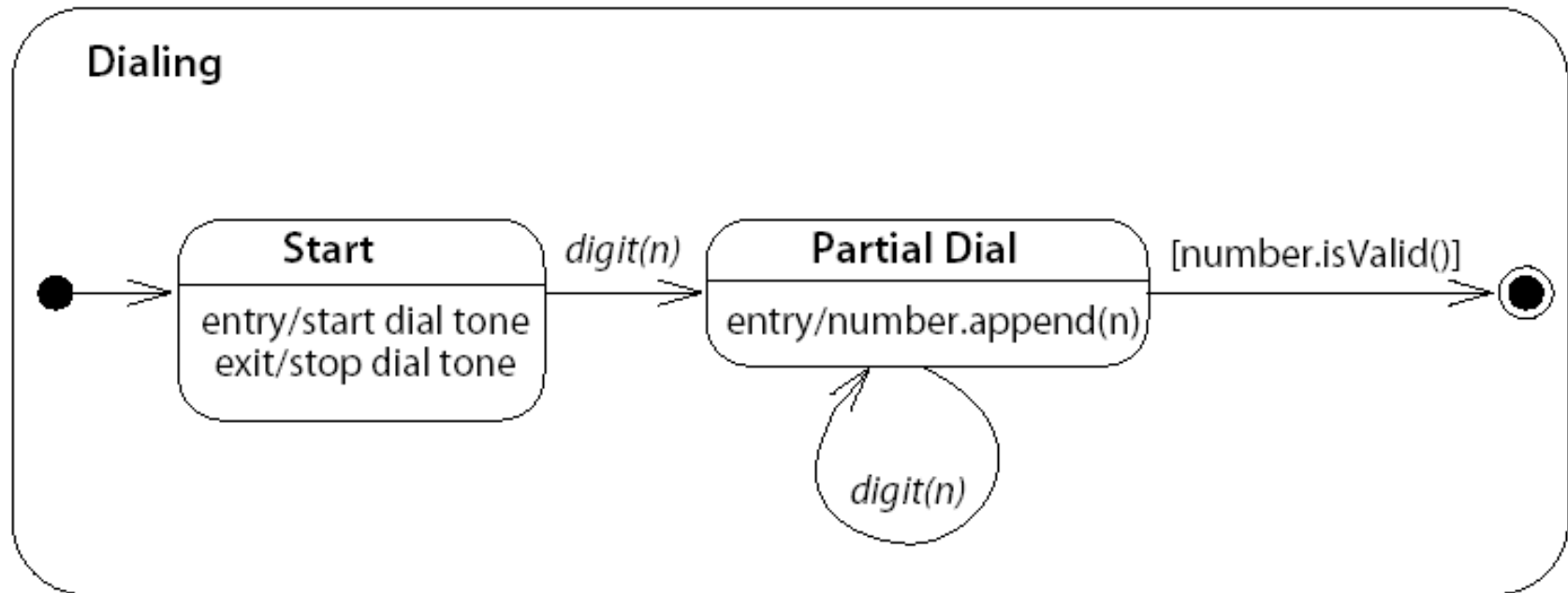
Композитные состояния и регионы

Композитное состояние (*composite state*) – состояние, содержащее в своем составе один регион или несколько ортогональных регионов.

Регион (*region*) – специальный элемент модели, который содержит состояния и переходы, и является частью композитного состояния или конечного автомата.

Ортогональное (*orthogonal*) композитное состояние – композитное состояние, содержащее более одного региона, которые в этом случае называются *ортогональными регионами* (*orthogonal regions*).

Простое композитное состояние

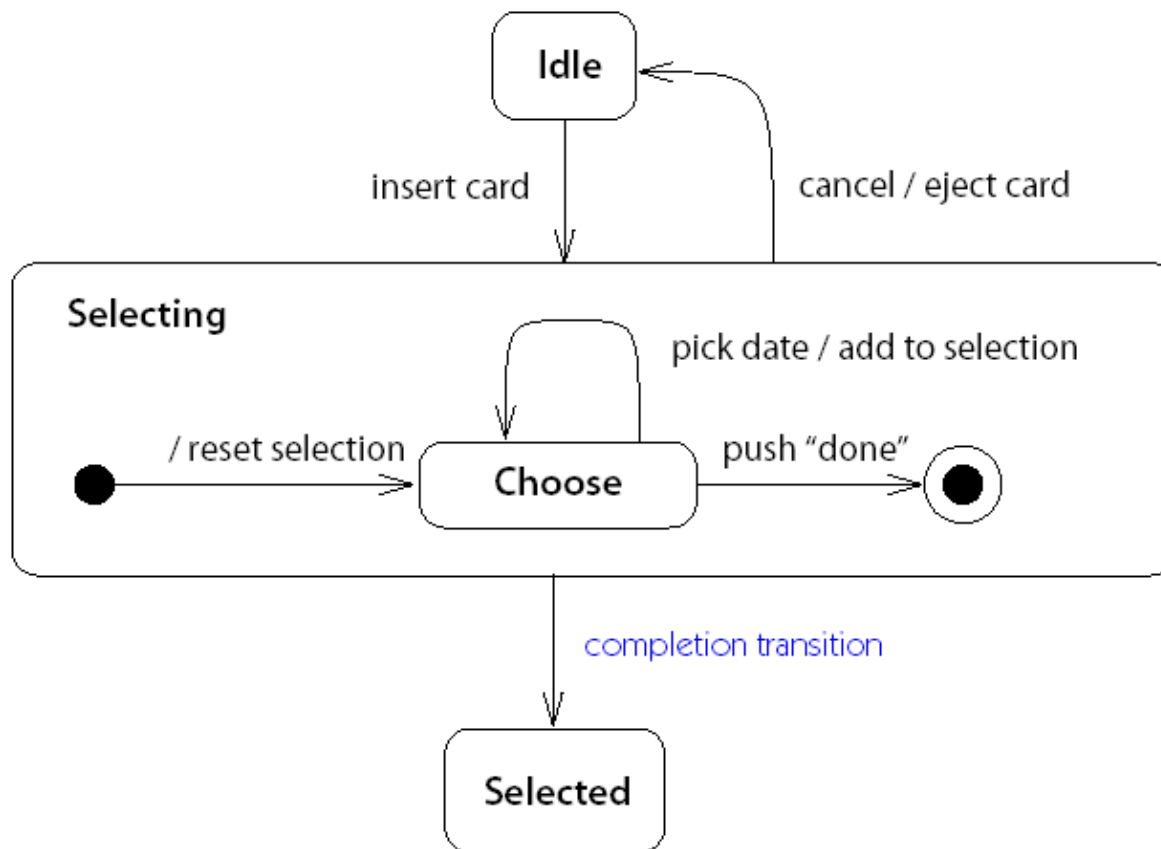


Любое состояние, заключенное в регион композитного состояния, называется **подсостоянием** (*substate*) этого композитного состояния.

Оно называется **прямым подсостоянием** (*direct substate*), если оно не содержится в никаком другом состоянии, иначе оно называется **непрямым подсостоянием** (*indirect substate*).

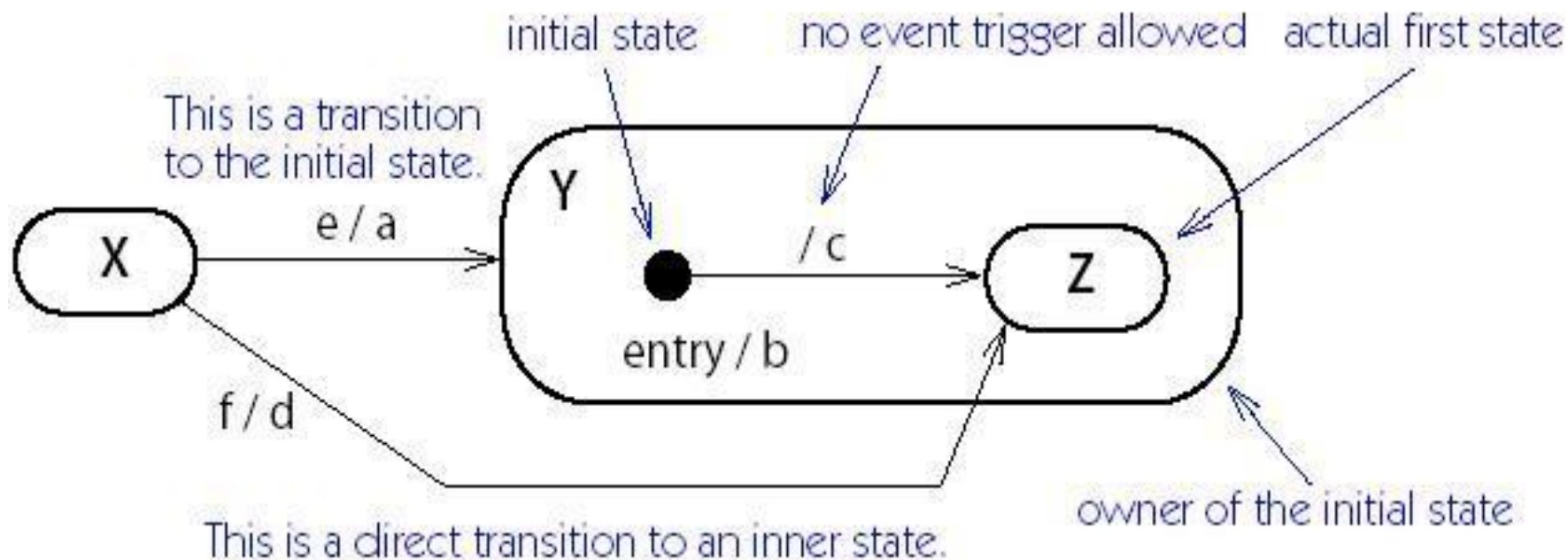
Вход в простое композитное состояние

Вход по умолчанию. Графически это изображается некоторым входящим переходом, который оканчивается на границе символа данного композитного состояния



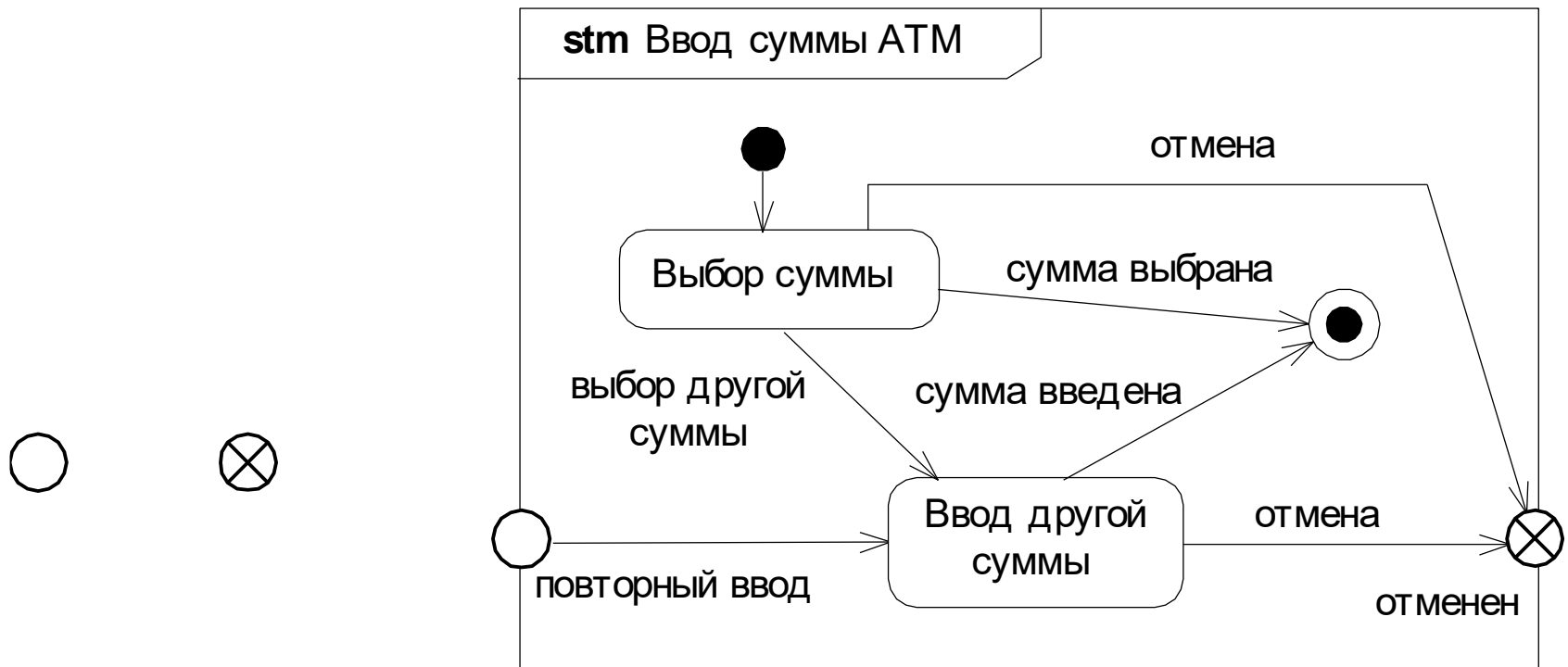
Вход в простое композитное состояние

Явный вход. Если переход входит в некоторое подсостояние простого композитного состояния, то такое подсостояние становится активным, а его входное действие выполняется после выполнения входного действия композитного состояния.



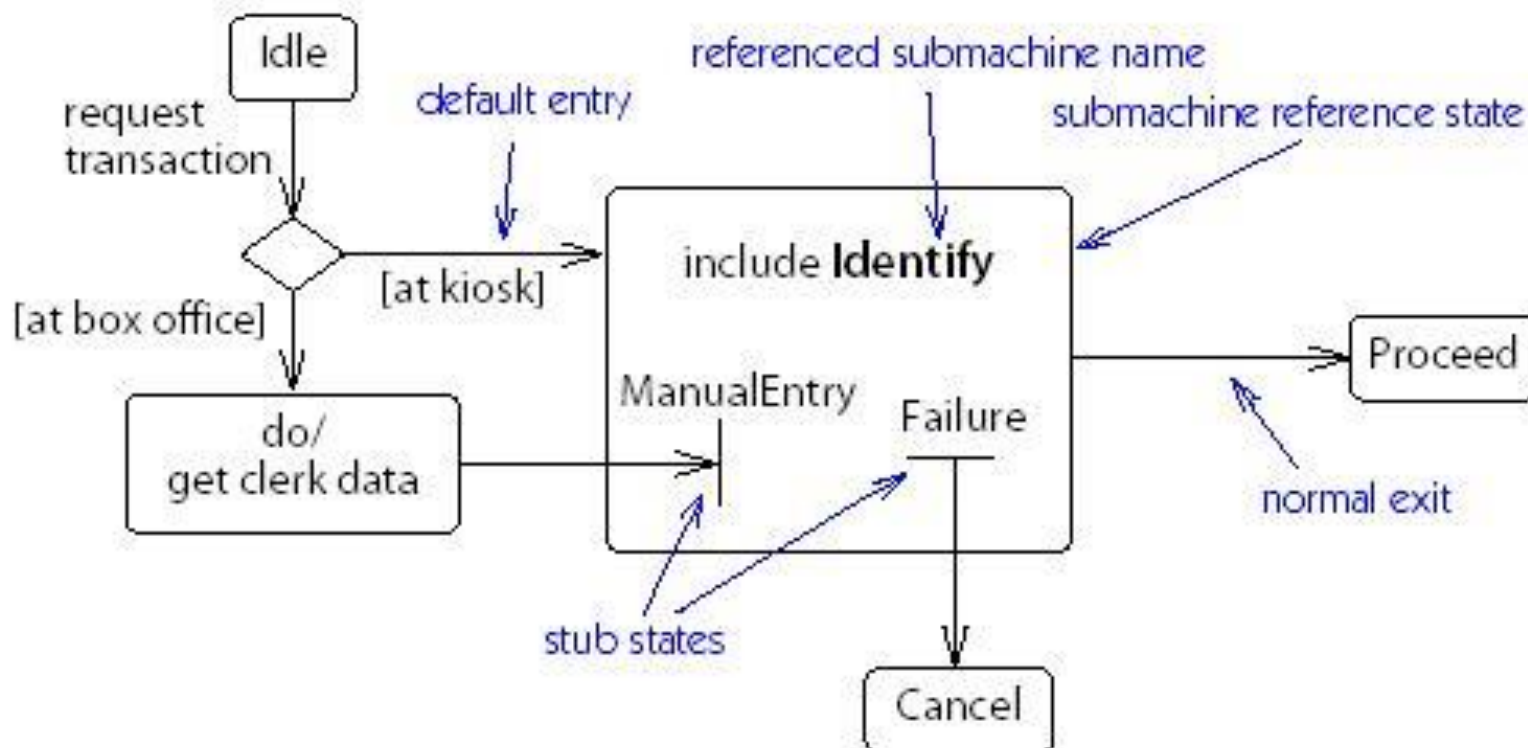
Вход в простое композитное состояние

Вход в точку входа. Если переход входит в простое композитное состояние через точку входа, то выполняется входное поведение композитного состояния до действия, ассоциированного с тем внутренним переходом, который выходит из этой точки входа.



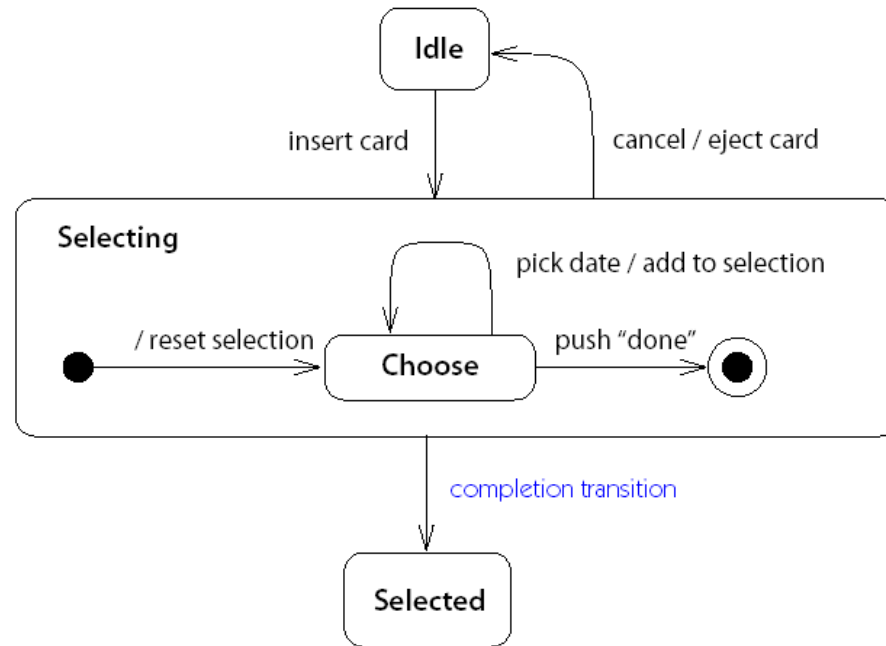
Выход из композитного состояния

Выход по умолчанию графически изображается некоторым выходящим переходом, который берет начало на границе символа простого композитного состояния и не содержит имени триггерного события.



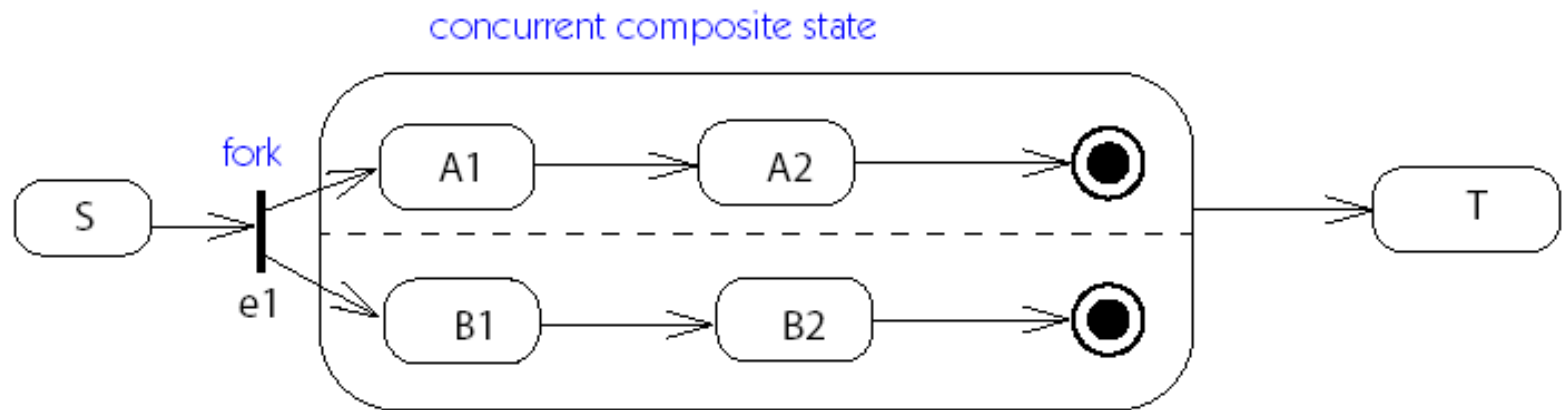
Выход из композитного состояния

Явный выход. Графически это также изображается выходящим переходом, который берет начало на границе символа простого композитного состояния и содержит имя некоторого триггерного события. Такой переход также называют *внешним* (external) переходом.

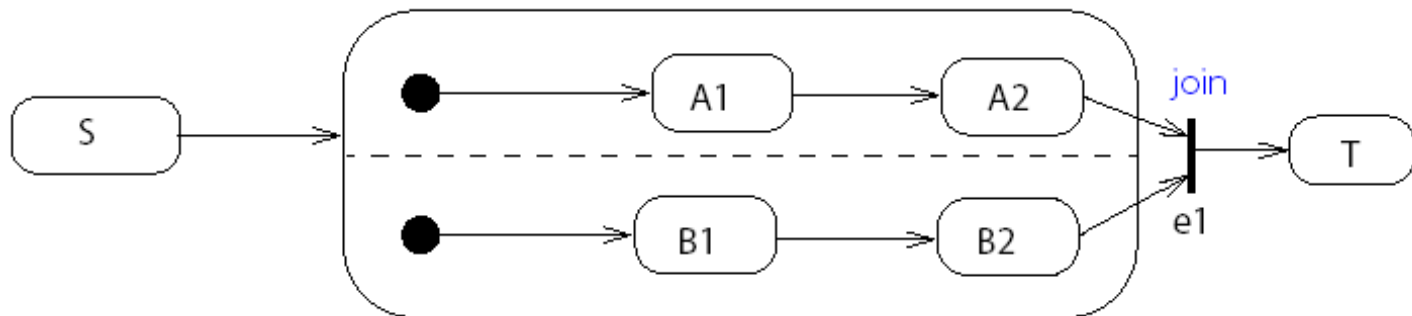


Ортогональное композитное состояние

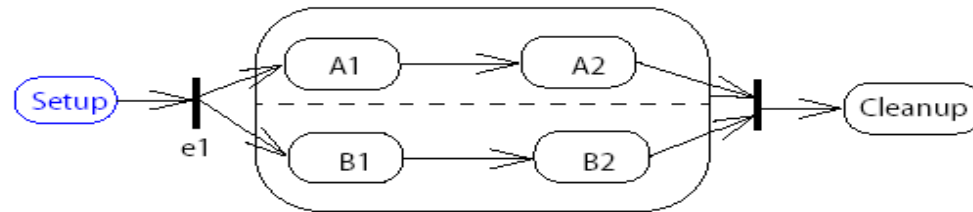
– композитное состояние, содержащее более одного региона называется **ортогональным** (*orthogonal*)



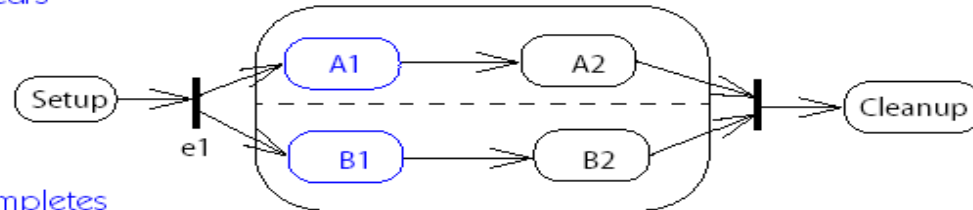
When *e1* occurs, *A1* and *B1* become active.



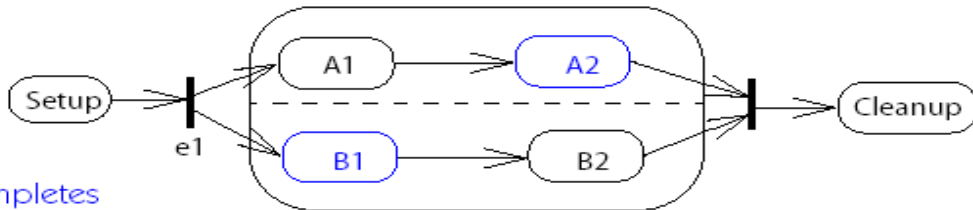
Изменение ортогональных подсостояний



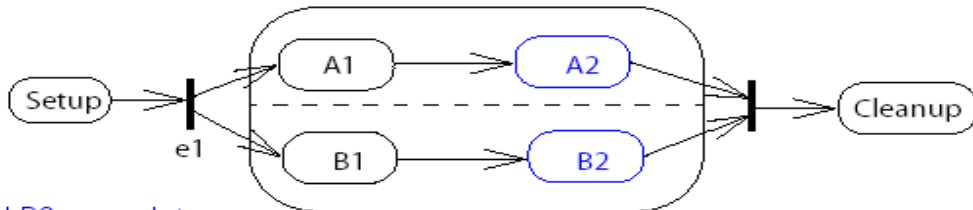
e1 occurs



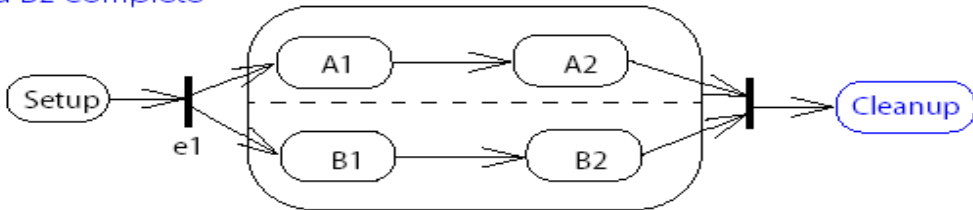
A1 completes



B1 completes



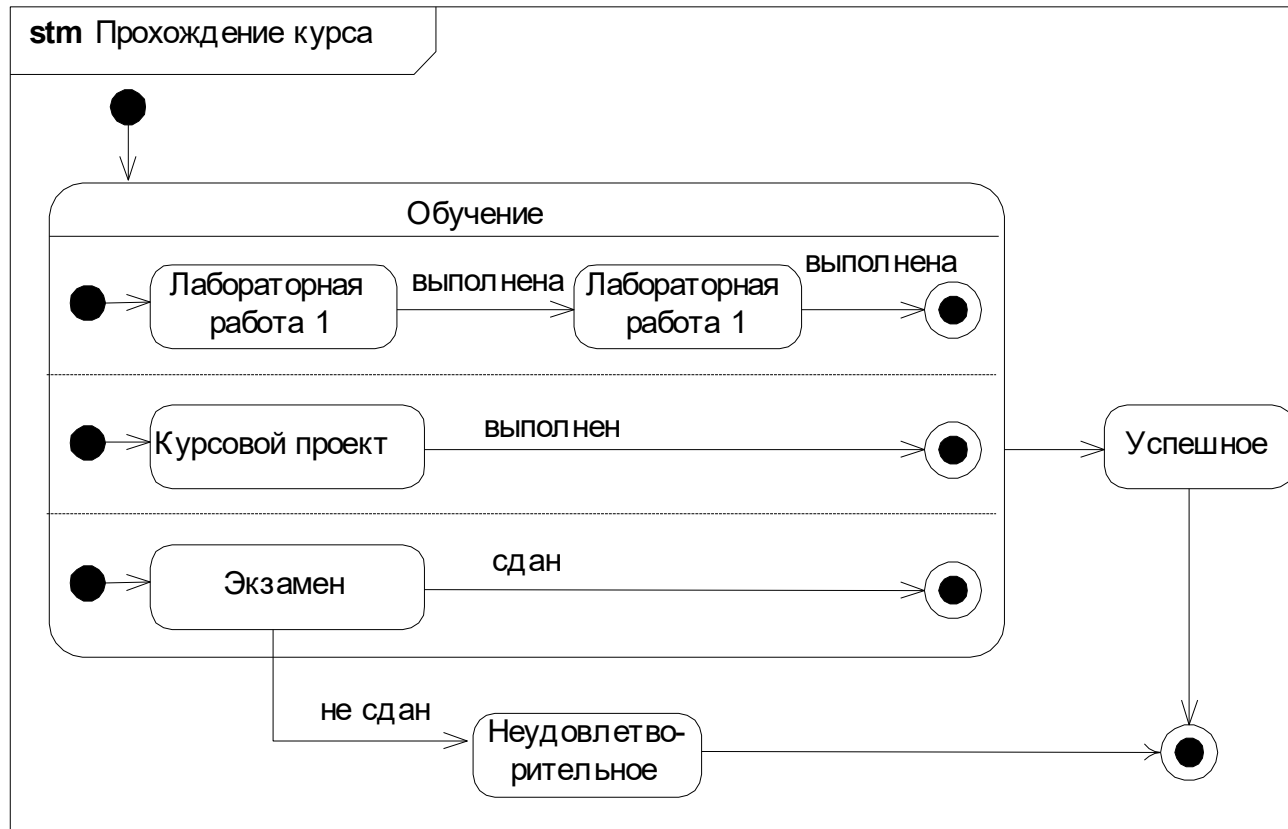
A2 and B2 complete



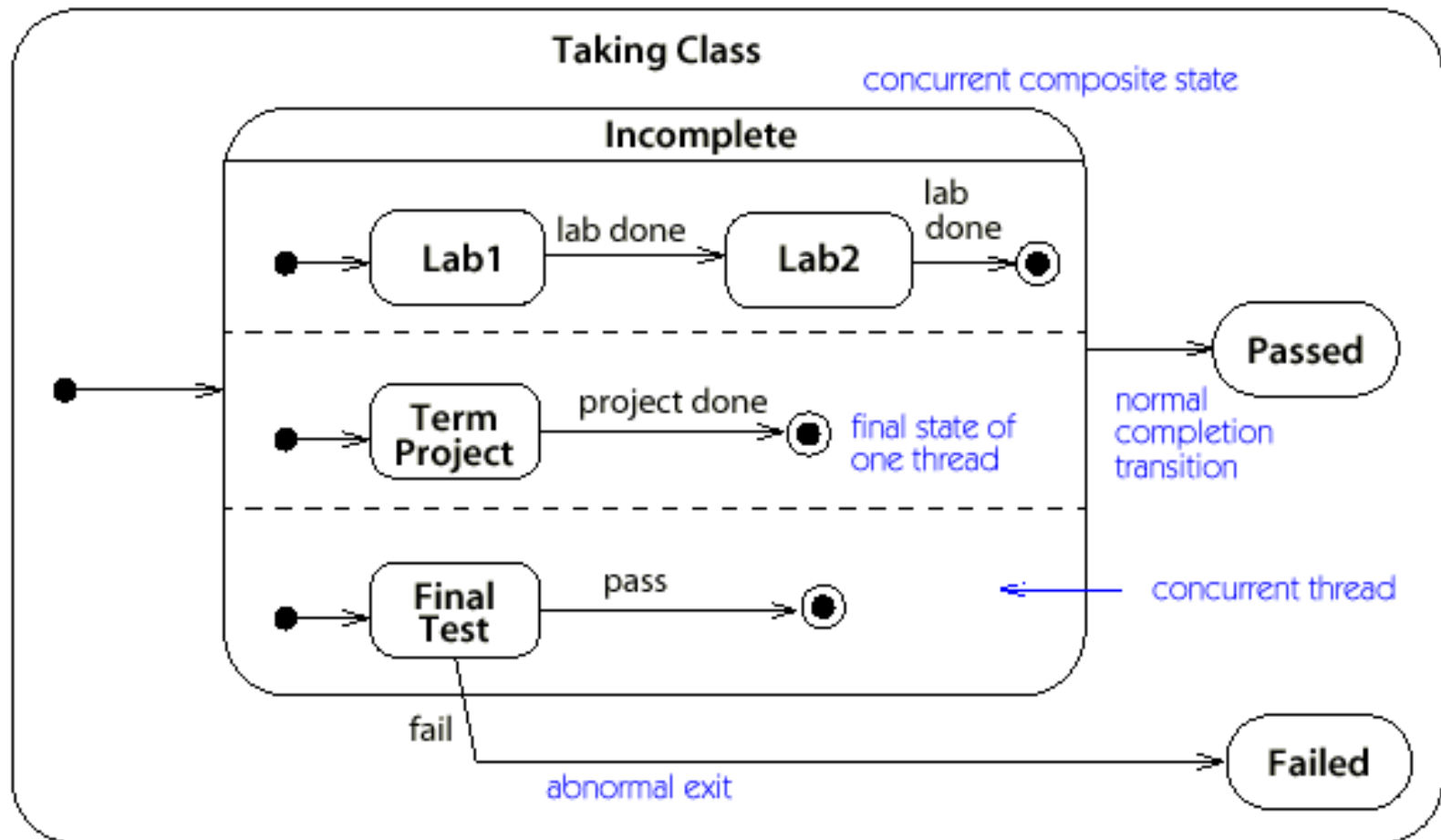
(Active states shown in blue)

Вход / выход в ортогональное композитное состояние

Вход и выход из ортогонального композитного состояния осуществляются в соответствии с правилами входа и выхода простого композитного состояния.



Исключительный выход из ортогонального подсостояния



Конечный автомат с параллельным композитным состоянием

Передача и прием сигнала

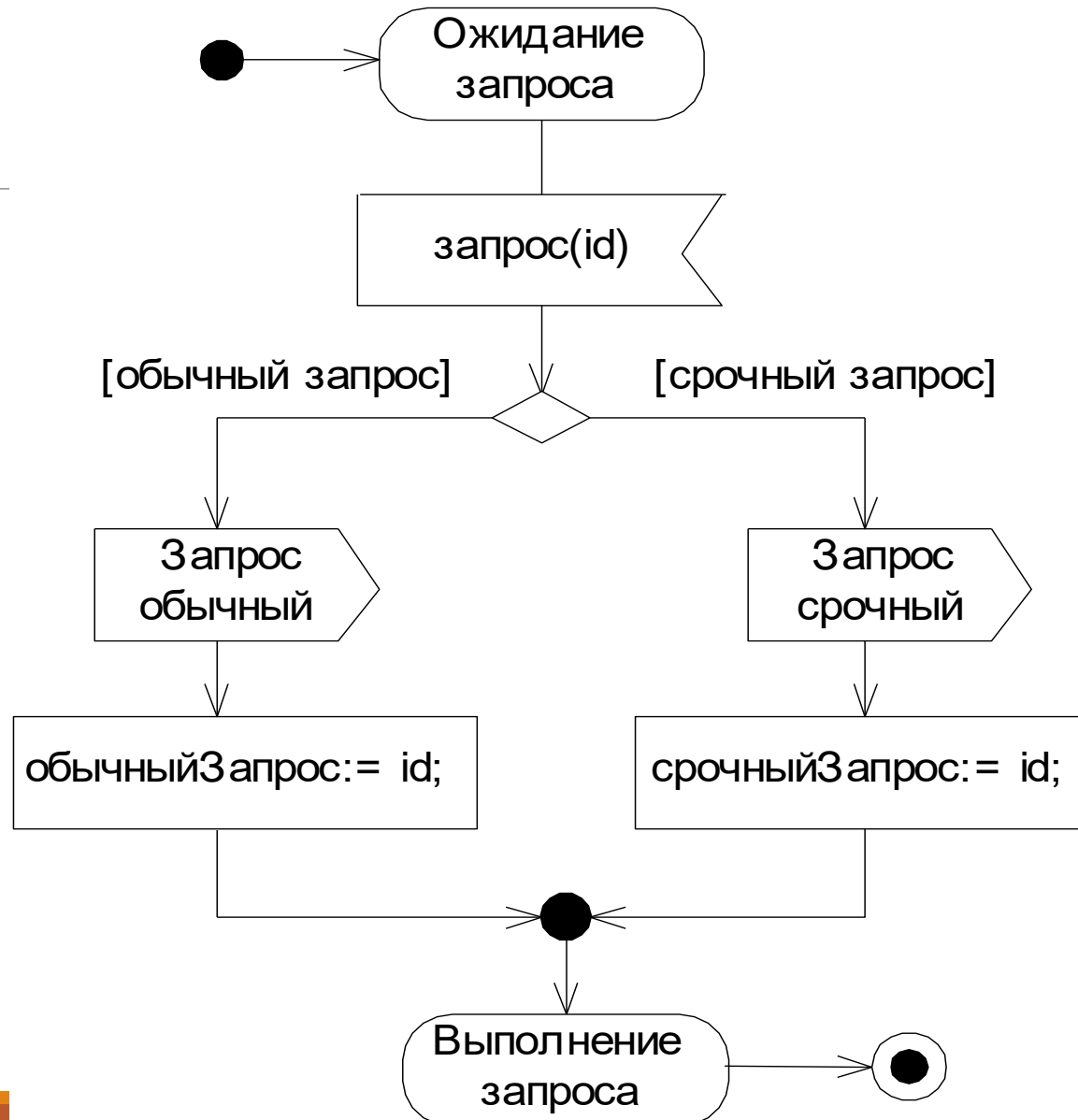
Передача сигнала является действием, которое имеет специальную графическую нотацию, аналогичную используемой на диаграммах деятельности

Прием сигнала, который также называют *приемом триггера*, является действием, которое имеет специальную графическую нотацию, используемую также на диаграммах деятельности

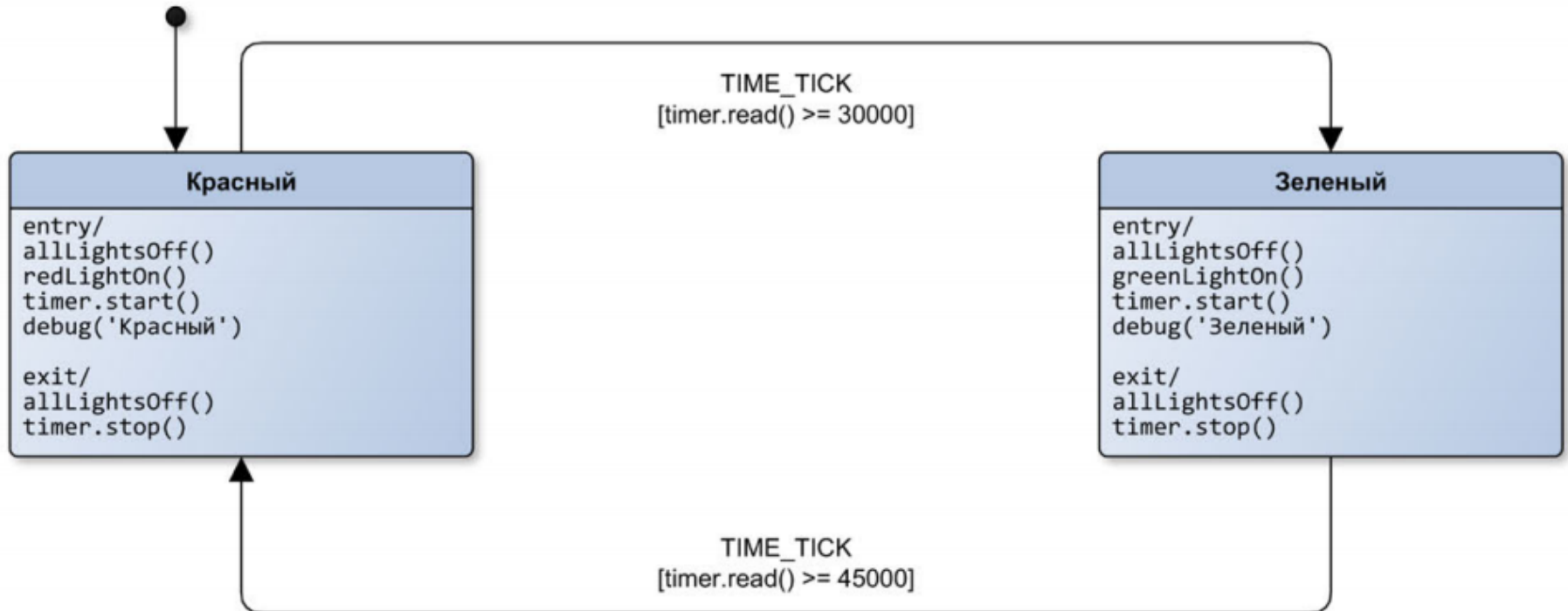
Синтаксис приема сигнала:

$\langle \text{прием-сигнала} \rangle :: = \langle \text{триггер} \rangle [\text{ ; } \text{ ; } \langle \text{триггер} \rangle]^* [[\text{ ' } \langle \text{ограждающее-условие} \rangle \text{ ' }]]$

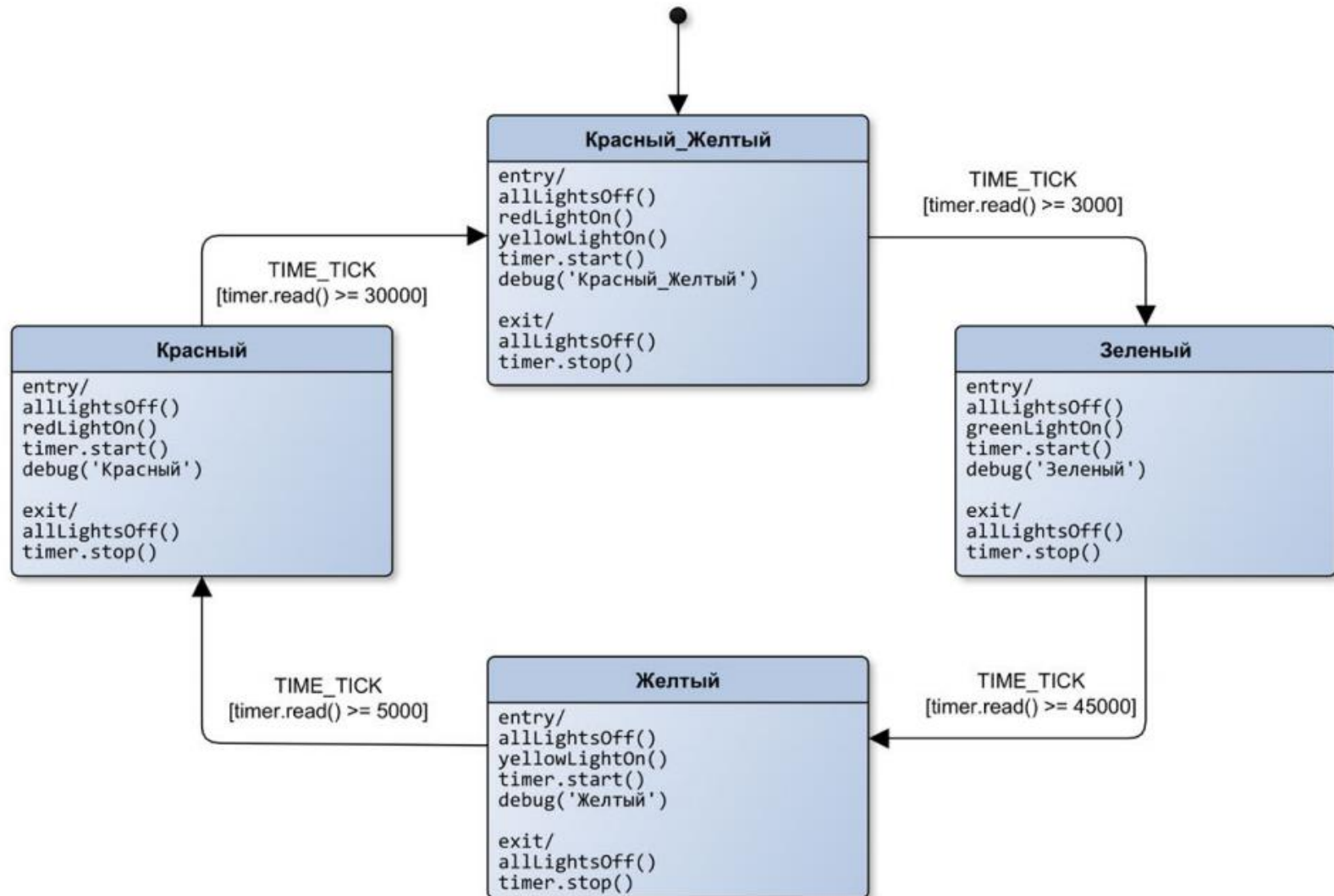
Пример приема и передачи сигнала



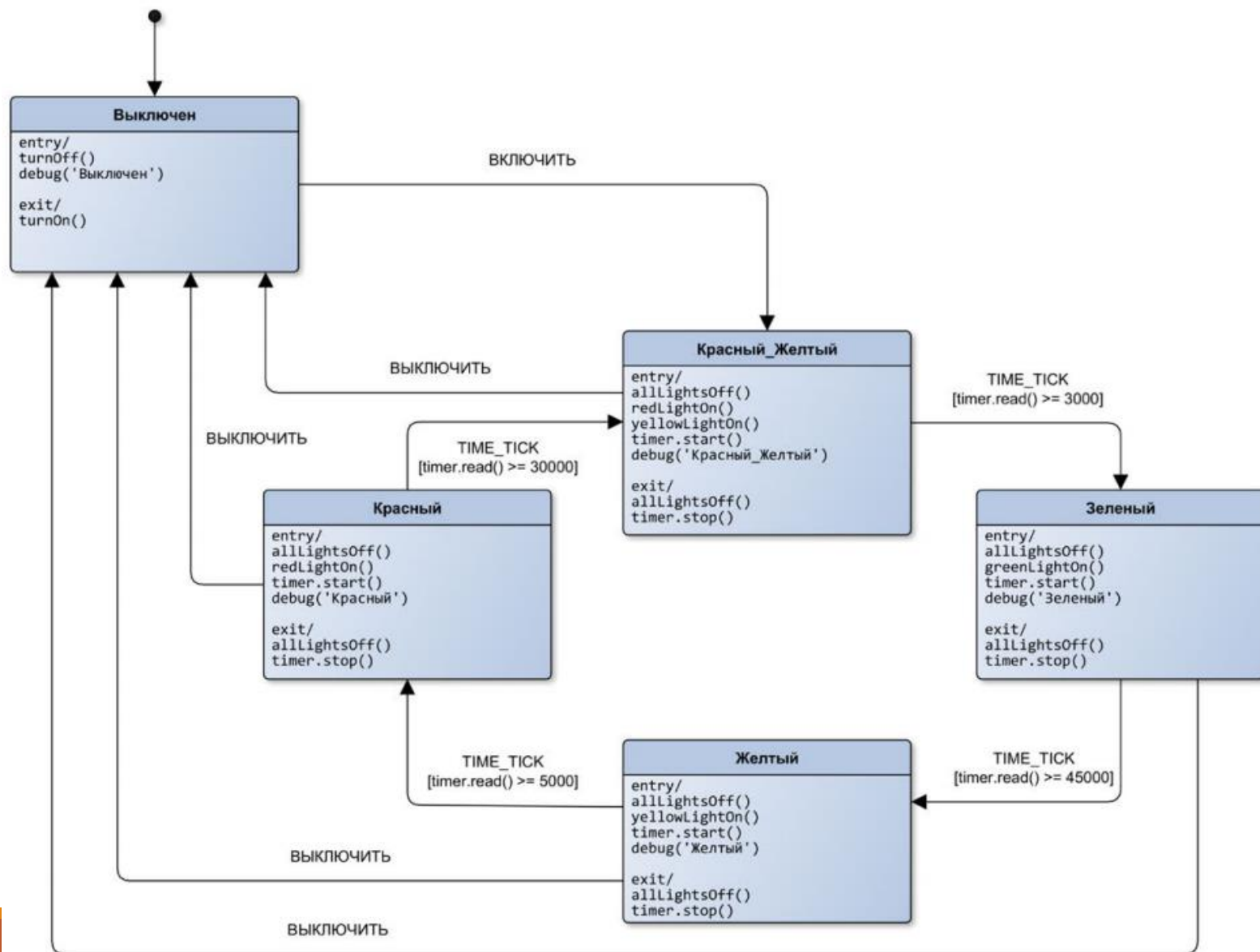
Пример: машина состояний для двухсекционного светофора



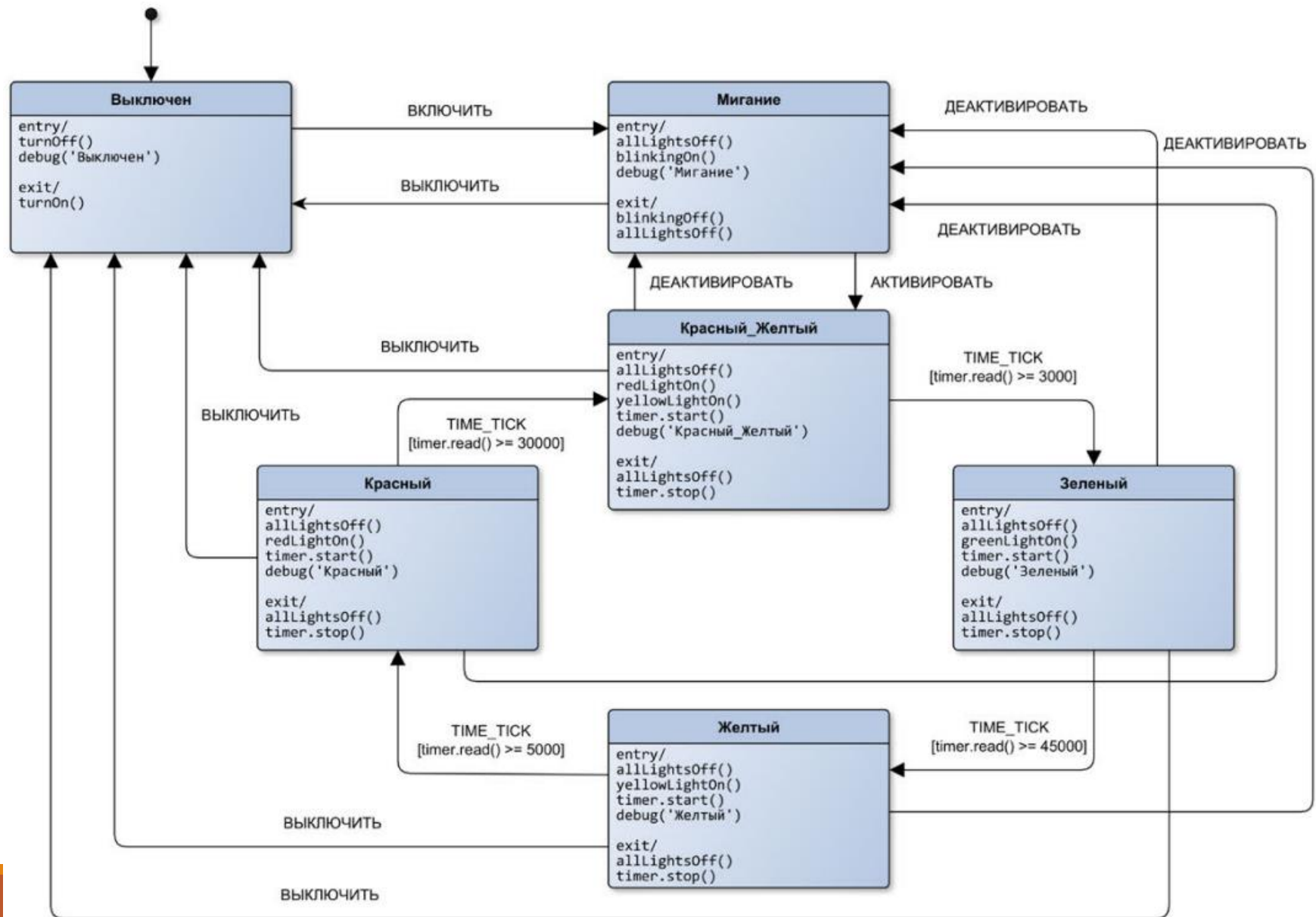
Пример: машина состояний для трехсекционного светофора



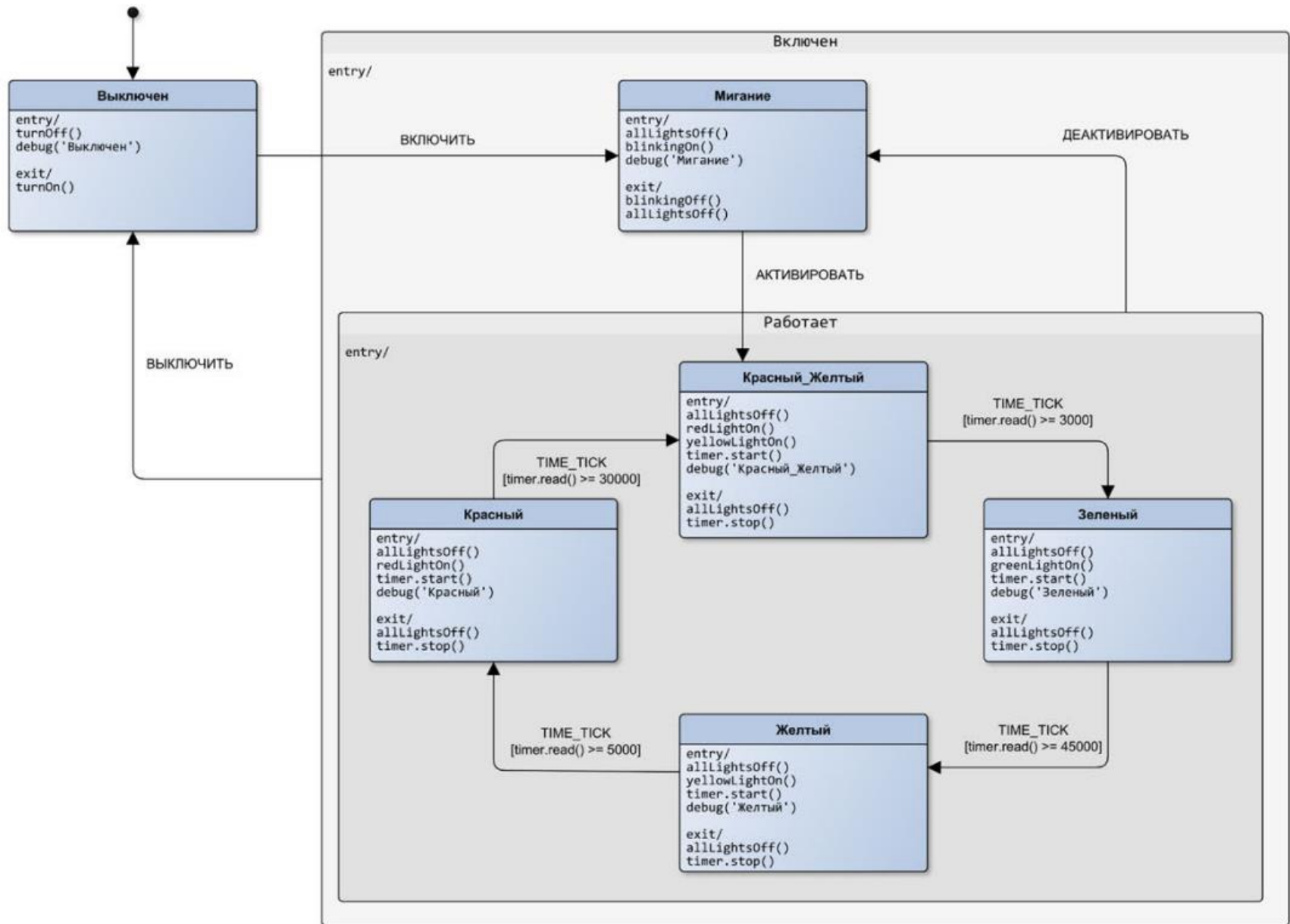
Машина состояний для трехсекционного светофора с режимом включить/выключить



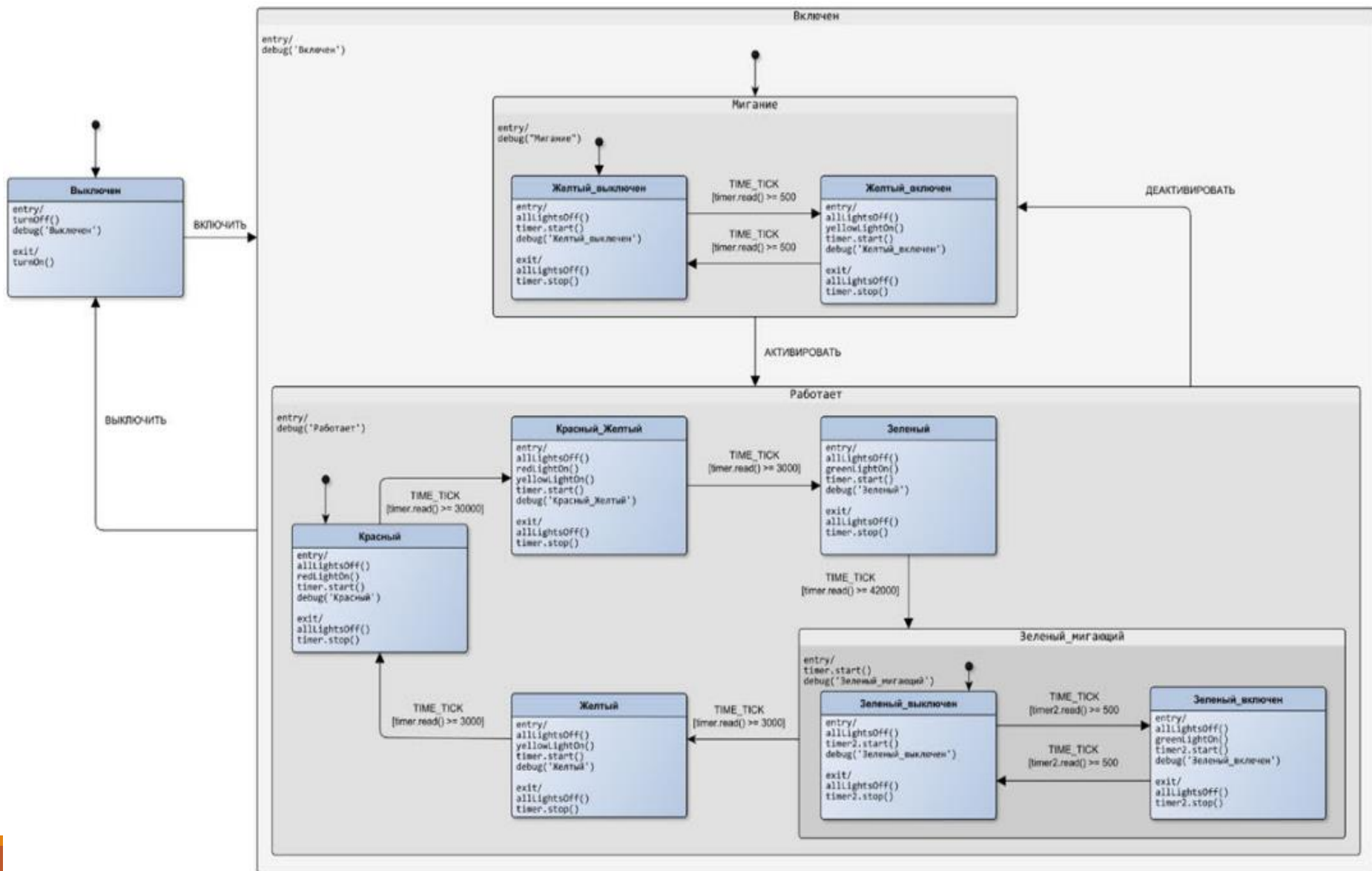
Плоская машина состояний для светофора с режимами включить/выключить и мигание



Иерархическая машина состояний для светофора с режимами включить/выключить и мигание



Иерархическая машина состояний для светофора с режимами включить/выключить и мигание желтого/зеленого



stateDiagram-v2

turnoff:<u>Выключен</u>
entry /turnOff
exit /turnOn

ВКЛ : Включен

[*] --> turnoff

turnoff --> ВКЛ :включить

ВКЛ --> turnoff :выключить

note left of turnoff

РИМС Светофор

end note

state ВКЛ {

[*] --> мигание

мигание : <u>Мигание</u>
entry /мигВкл
exit /мигВыкл

мигание --> Работает :активировать

Работает --> мигание :остановить

state Работает {

direction LR

[*] --> желтыйК

greens: <u>Зеленый</u>
entry /зеленВкл
exit /зеленВыкл

желтый: <u>Желтый</u>
entry /желтВкл
exit /желтВыкл

красный: <u>Красный</u>
entry /красВкл
exit /красВыкл

желтыйК: <u>Желтый</u>
entry /желтВкл
exit /желтВыкл

желтыйК --> greens :[+7сек]

greens --> желтый :[+50сек]

желтый --> красный :[+7сек]

красный --> желтыйК :[+40сек]

} }

Mermaid
state diagram

